# Ambulancias Cuánticas

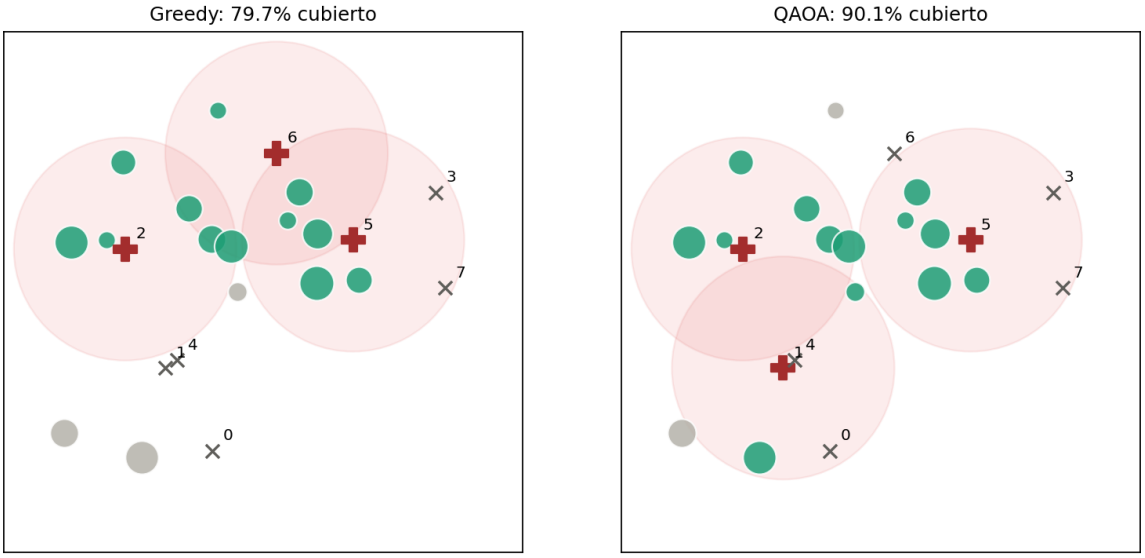
Ubicar bases de emergencia con QAOA · Hackathon Qollab x IonQ · Tema: Quantum Impact

## La idea en 30 segundos

Una ciudad tiene pocas ambulancias y muchos lugares posibles para ubicarlas. Elegir bien dónde ponerlas decide cuánta gente recibe ayuda en pocos minutos. El proyecto modela ese problema, lo resuelve con un método clásico rápido (greedy), con la solución exacta y con **QAOA**, un algoritmo cuántico de optimización, y compara los resultados.

| Método | Bases elegidas | Población cubierta |
|---|---|---|
| Greedy (clásico) | 2, 5, 6 | 1.845 mil (79,7%) |
| Exacta (fuerza bruta) | 1, 2, 5 | 2.086 mil (90,1%) |
| QAOA (cuántico) | 1, 2, 5 | 2.086 mil (90,1%) |

En la instancia de ejemplo, greedy deja sin cubrir a 241 mil personas más que la solución óptima. QAOA encuentra el óptimo, y además lo mide el **27,5%** de las veces, contra un 3,6% si se eligiera una combinación al azar (7,7 veces mejor). El 100% de sus mediciones usa exactamente 3 ambulancias.




Verde: zona cubierta (tamano = poblacion) · Gris: sin cubrir · Cruz roja: base elegida

Resultados verificados al correr el código del anexo con Qiskit 2.5 en simulador (statevector, 4096 mediciones). Los números pueden variar un poco en cada corrida.

# 1. El problema y por qué importa

En una emergencia cardíaca, cada minuto sin atención baja las chances de sobrevivir. Por eso los sistemas de emergencia buscan que la mayor cantidad de gente quede a pocos minutos de una ambulancia. Con recursos limitados, la pregunta es: **¿en cuáles de los sitios posibles conviene ubicar las K bases disponibles?**

Es un problema de **máxima cobertura**, uno de los clásicos de la investigación operativa. El mismo modelo sirve para bomberos, centros de vacunación, antenas de celular o cargadores de autos eléctricos, y ese es un buen argumento de impacto para los jueces.

### Por qué es difícil

La cantidad de combinaciones crece muy rápido. Con 8 sitios y 3 ambulancias hay 56 opciones, pero con 100 sitios y 10 ambulancias hay más de 17 billones ($17 \times 10^{12}$). Probar todo deja de ser posible, y los métodos rápidos como greedy no garantizan el óptimo: en el ejemplo, greedy elige primero el sitio que más gente cubre solo, y esa decisión temprana lo encierra en una solución peor.

# 2. Por qué usar computación cuántica (con honestidad)

QAOA prepara una superposición de muchas combinaciones a la vez y, capa por capa, hace que las mejores tengan más probabilidad de salir al medir. Es un método **híbrido**: un optimizador clásico ajusta los ángulos del circuito y el circuito cuántico genera las soluciones.

Lo que hay que decir con claridad en el video y en la página:

- Con 8 qubits, la fuerza bruta clásica resuelve el problema al instante. Esta instancia sirve para **demostrar que el método funciona**, no para ganarle en velocidad a una computadora normal.
- Hoy no hay una ventaja cuántica demostrada para este tipo de problemas a gran escala. QAOA es una de las candidatas más estudiadas, y por eso vale la pena medirla.
- Lo que sí se mide en el proyecto: calidad de la solución contra greedy, y cuánto amplifica QAOA la probabilidad de la mejor respuesta frente al azar.

Los jueces valoran más la honestidad y una buena medición que una promesa exagerada.

# 3. Cómo funciona la solución

| Paso | Qué hace | Dónde está en el código |
|---|---|---|
| 1. Datos | Zonas con población y sitios candidatos con coordenadas. | ZONAS, SITIOS |
| 2. Cobertura | Una zona queda cubierta si está a menos de 3 km (unos 4-5 minutos) de una base. | CUBRE |
| 3. Líneas base | Greedy y fuerza bruta, para comparar. | greedy(), exacta() |
| 4. Costo | Convierte el problema en una función cuadrática de variables 0/1. | Q, costo() |
| 5. Circuito | Estado inicial de Dicke + capas de costo + mezclador XY. | circuito_qaoa() |
| 6. Optimización | COBYLA ajusta los ángulos usando simulación local. | optimizar() |
| 7. Medición | Un solo job de 4096 mediciones y se elige la mejor combinación medida. | muestrear() |

# 4. La formulación, paso a paso

Cada sitio candidato es un qubit: $x_i = 1$ si se pone una base en el sitio i. El objetivo es minimizar:

$$\textbf{costo(x)} = -\Sigma_i\, w_i\, x_i + \Sigma_{i<j}\, o_{ij}\, x_i\, x_j, \text{ con la condición } \Sigma\, x_i = K$$

- $w_i$ es la población que cubre el sitio i por sí solo (se resta porque queremos maximizarla).
- $o_{ij}$ es la población que cubren los sitios i y j al mismo tiempo. Se suma para no contar dos veces a la misma gente.
- Es una aproximación de segundo orden de la cobertura real. Por eso, al final, cada combinación medida se evalúa con la cobertura **real** y se elige la mejor.

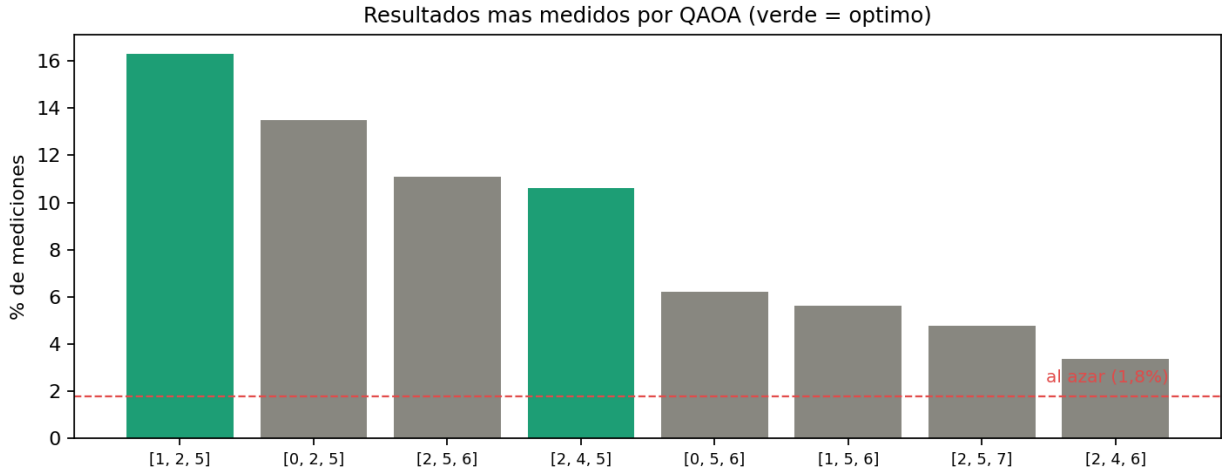
### La restricción de K ambulancias, sin penalidades

La forma típica de imponer "exactamente K bases" es sumar una penalidad al costo. La probamos y funcionó mal: solo el 40% de las mediciones respetaba la restricción y el óptimo salía menos que al azar. La versión final usa dos ideas mejores:

- **Estado inicial de Dicke:** una superposición pareja de las 56 formas de elegir 3 sitios.
- **Mezclador XY:** en lugar de prender y apagar qubits sueltos, "mueve" una ambulancia de un sitio a otro. Así la cantidad de bases nunca cambia y el 100% de las mediciones es válida.

Ese cambio es un muy buen momento para el video: muestra que entendiste el problema y no solo copiaste un ejemplo.

# 5. Resultados




- QAOA encuentra el óptimo (90,1% de la población) donde greedy se queda en 79,7%.
- Hay dos soluciones óptimas empatadas, [1, 2, 5] y [2, 4, 5], porque los sitios 1 y 4 están casi en el mismo lugar. QAOA da alta probabilidad a las dos.
- Probabilidad total de medir un óptimo: 27,5%, contra 3,6% al azar.
- Parámetros: 8 qubits, 3 capas (6 ángulos), radio de 3 km, 3 ambulancias.

### Experimentos para sumar a la página

- Probabilidad del óptimo según la cantidad de capas (1 a 5). Ya probamos 3 y 4: entre 26% y 28%.
- Repetir con 10 a 20 instancias aleatorias y contar en cuántas greedy falla y QAOA encuentra el óptimo.
- Variar el radio o la cantidad de ambulancias y mostrar cómo cambia la cobertura.

## 6. Checklist de requisitos de Qollab

| Requisito | Cómo lo cumple el proyecto |
|---|---|
| Proyecto en Qiskit que corre en los simuladores gratuitos de Qollab | Usa solo Qiskit, NumPy y SciPy. Verificar que esas librerías estén disponibles en Qollab. |
| Publicado y ejecutable en Qollab, licencia MIT | Subir el archivo, agregar licencia MIT y presionar Run antes de enviar. |
| Un job por ejecución en IonQ | Los ángulos se optimizan en simulación local y luego se manda un único job de medición. |
| Página del proyecto que explique qué hace y cómo correrlo | Ver la sección 8. |
| Video de 2 a 3 minutos en un enlace público | Ver la sección 9. YouTube, Loom, Vimeo o Google Drive sin inicio de sesión. |
| Declarar código previo o ayuda externa | Indicar en la página que el código se generó con ayuda de IA durante el hackathon. |

Ojo con dos cosas técnicas: **initialize()** (el estado de Dicke) se traduce a muchas compuertas; en simulador no hay problema, pero conviene probarlo en el simulador de IonQ. Y si matplotlib no está disponible en Qollab, los gráficos se generan aparte y se suben como imágenes a la página.

## 7. Plan hasta el cierre

El envío cierra el **domingo 11 de octubre a las 19:00 ET, que son las 20:00 en Argentina**. No dejes el envío para el último minuto.

| Cuándo | Tarea | Resultado |
|---|---|---|
| Sábado, primeras 3 horas | Correr el código, entender cada sección y subirlo a Qollab. Presionar Run en el simulador gratuito y en el de IonQ. | Proyecto publicado que corre |
| Sábado, tarde | Opcional: reemplazar los datos de ejemplo por datos reales, por ejemplo barrios con población y hospitales de los datos abiertos de Buenos Aires. Correr los experimentos extra. | Resultados con datos reales |
| Sábado, noche | Escribir la página del proyecto con las secciones de abajo y los gráficos. | Página completa |
| Domingo, mañana | Grabar el video con el guion de la sección 9 y subirlo con enlace público. | Video listo |
| Domingo, antes de las 18:00 ART | Revisar el checklist y que el capitán envíe desde la página de registro. | Proyecto enviado |

## 8. Estructura de la página del proyecto

- **Título y frase:** "Ambulancias Cuánticas: ubicar bases de emergencia con QAOA".
- **El problema:** dos o tres frases sobre tiempos de respuesta y recursos limitados.
- **La solución:** el pipeline de la sección 3 y la idea del mezclador XY en palabras simples.
- **Resultados:** la tabla de métodos, el mapa y el gráfico de barras.
- **Cómo correrlo:** presionar Run; qué imprime y cuánto tarda. Cómo cambiar K, el radio o los datos.
- **Limitaciones y próximos pasos:** la sección 10, con honestidad.
- **Créditos:** qué existía antes, herramientas de IA usadas, licencia MIT.

## 9. Guion del video (2:30)

| Tiempo | Qué se ve | Qué decís |
|---|---|---|
| 0:00-0:20 | Mapa con zonas grises. | "Si tenés solo 3 ambulancias para toda una ciudad, ¿dónde las ponés? De eso depende cuánta gente recibe ayuda a tiempo." |
| 0:20-0:45 | Mapa de greedy. | "Lo intuitivo es poner la primera donde más gente hay. Ese método se llama greedy y cubre al 79,7%. Pero no es lo mejor." |
| 0:45-1:30 | El código y el circuito. | "Convertí el problema en uno que entiende una computadora cuántica: cada sitio es un qubit. Usé QAOA, y en vez de penalidades usé un mezclador XY que mueve ambulancias entre sitios sin cambiar cuántas hay." |
| 1:30-2:05 | Presionar Run en Qollab y mostrar el gráfico de barras. | "QAOA encuentra el óptimo: 90,1%, 241 mil personas más que greedy. Y lo mide 7,7 veces más seguido que el azar." |
| 2:05-2:30 | Mapa de QAOA. | "Con 8 qubits esto también se resuelve clásicamente. El objetivo es medir cómo se comporta QAOA y preparar el camino para ciudades reales cuando el hardware crezca." |

## 10. Limitaciones y próximos pasos

- **Escala:** un qubit por sitio. Hoy se simulan con comodidad unos 20 a 25 qubits.
- **Optimización de ángulos:** se hace con simulación exacta, que no escala. A mayor escala habría que optimizar en hardware o reutilizar ángulos de problemas chicos.
- **Modelo simplificado:** distancia en línea recta en lugar de tiempo real por calles, y demanda fija.
- **Próximos pasos:** datos reales de la ciudad, tiempos de viaje por calles, demanda que cambia según la hora y prueba en hardware de IonQ.

# Anexo: código completo (ambulancias_qaoa.py)

Probado con Qiskit 2.5.2, NumPy y SciPy. Tarda alrededor de 1 a 2 minutos en simulador. También se entrega como archivo .py aparte.

```
"""
Ubicacion de ambulancias con QAOA (Qiskit)
Hackathon Qollab x IonQ - tema Quantum Impact

Problema: elegir K bases entre N sitios candidatos para cubrir a la mayor
cantidad de poblacion dentro de un radio (tiempo de respuesta).
Se compara: greedy clasico, solucion exacta (fuerza bruta) y QAOA.

QAOA usa un mezclador XY y empieza en un estado de Dicke, asi que todas
las mediciones usan exactamente K ambulancias (no hace falta penalidad).
"""
import itertools, math
import numpy as np
from scipy.optimize import minimize
from qiskit import QuantumCircuit
from qiskit.circuit import ParameterVector
from qiskit.quantum_info import Statevector
from qiskit.primitives import StatevectorSampler

# ---------------- 1. Datos ----------------
# Zonas: (x km, y km, poblacion en miles). Reemplazar por datos reales.
ZONAS = [(1.78, 7.39, 63), (6.65, 7.93, 66), (4.61, 7.41, 164), (6.98, 8.69, 159),
         (7.44, 6.24, 245), (2.22, 9.49, 133), (2.72, 1.54, 227), (0.63, 2.21, 167),
         (5.30, 6.01, 76), (8.57, 6.31, 145), (7.45, 7.56, 191), (0.82, 7.34, 232),
         (4.00, 8.25, 150), (5.13, 7.23, 235), (4.78, 10.89, 62)]
# Sitios candidatos para bases (x km, y km)
SITIOS = [(4.63, 1.72), (3.37, 3.95), (2.28, 7.15), (10.65, 8.67),
          (3.67, 4.16), (8.42, 7.39), (6.36, 9.73), (10.90, 6.11)]
RADIO_KM = 3.0   # a 40 km/h, 3 km son unos 4-5 minutos de respuesta
K = 3            # ambulancias disponibles
N = len(SITIOS)  # un qubit por sitio candidato
POB_TOTAL = sum(z[2] for z in ZONAS)

# ---------------- 2. Cobertura ----------------
CUBRE = [{z for z, (x, y, _) in enumerate(ZONAS) if math.dist((x, y), s) <= RADIO_KM}
         for s in SITIOS]

def poblacion_cubierta(sel):
    zonas = set().union(*[CUBRE[i] for i in sel]) if sel else set()
    return sum(ZONAS[z][2] for z in zonas)

# ---------------- 3. Lineas base clasicas ----------------
def greedy():
    """Agrega en cada paso el sitio que mas poblacion nueva cubre."""
    sel = []
    for _ in range(K):
        mejor = max((i for i in range(N) if i not in sel),
                    key=lambda i: poblacion_cubierta(sel + [i]))
        sel.append(mejor)
    return sorted(sel)

def exacta():
    """Prueba todas las combinaciones (solo posible porque N es chico)."""
    return list(max(itertools.combinations(range(N), K),
                    key=lambda s: poblacion_cubierta(list(s))))

# ---------------- 4. Funcion de costo cuadratica ----------------
# Minimizar  -sum_i w_i x_i + sum_{i<j} o_ij x_i x_j
# w_i  = poblacion que cubre el sitio i
# o_ij = poblacion que cubren i y j a la vez (para no contarla dos veces)
w = np.array([sum(ZONAS[z][2] for z in CUBRE[i]) for i in range(N)]) / POB_TOTAL
Q = np.diag(-w)
for i, j in itertools.combinations(range(N), 2):
    Q[i, j] = sum(ZONAS[z][2] for z in CUBRE[i] & CUBRE[j]) / POB_TOTAL

def costo(x):
    x = np.array(x)
    return float(x @ Q @ x)

ESTADOS = [[(k >> i) & 1 for i in range(N)] for k in range(2 ** N)]
COSTOS = np.array([costo(x) for x in ESTADOS])
PESO = np.array([sum(x) for x in ESTADOS])

# ---------------- 5. Circuito QAOA ----------------
P = 3        # capas
ESCALA = 5.0 # escala del costo dentro del circuito

def estado_dicke():
    """Superposicion pareja de todas las formas de elegir K sitios."""
    v = (PESO == K).astype(float)
```

```
    return v / np.linalg.norm(v)

def circuito_qaoa(p=P):
    g, b = ParameterVector("g", p), ParameterVector("b", p)
    qc = QuantumCircuit(N)
    qc.initialize(estado_dicke())
    for capa in range(p):
        # Capa de costo: x_i = (1 - Z_i)/2  ->  rotaciones RZ y RZZ
        for i in range(N):
            vecinos = sum(Q[min(i, j), max(i, j)] for j in range(N) if j != i)
            qc.rz(-g[capa] * ESCALA * (Q[i, i] + vecinos / 2), i)
        for i, j in itertools.combinations(range(N), 2):
            if Q[i, j] != 0:
                qc.rzz(g[capa] * ESCALA * Q[i, j] / 2, i, j)
        # Capa de mezcla XY en anillo: mueve una ambulancia de un sitio
        # a otro sin cambiar cuantas hay
        for inicio in (0, 1):
            for i in range(inicio, N, 2):
                j = (i + 1) % N
                qc.rxx(b[capa], i, j)
                qc.ryy(b[capa], i, j)
    return qc

ansatz = circuito_qaoa()

def energia(params):
    probs = Statevector(ansatz.assign_parameters(params)).probabilities()
    return float(probs @ COSTOS)

def optimizar(intentos=4, semilla=1):
    """Optimiza los angulos en simulacion local (parte clasica del hibrido)."""
    rng = np.random.default_rng(semilla)
    mejor = None
    for t in range(intentos):
        if t == 0:   # arranque tipo "recocido": gamma crece, beta baja
            x0 = np.concatenate([np.linspace(0.1, 1, P), np.linspace(0.5, 0.05, P)])
        else:
            x0 = rng.uniform(0, 1.5, 2 * P)
        r = minimize(energia, x0, method="COBYLA", options={"maxiter": 500})
        if mejor is None or r.fun < mejor.fun:
            mejor = r
    return mejor.x

def bits_a_sitios(bits):
    # Qiskit ordena los bits al reves: el ultimo caracter es el qubit 0
    return [i for i in range(N) if bits[N - 1 - i] == "1"]

def muestrear(params, shots=4096):
    circ = ansatz.assign_parameters(params)
    circ.measure_all()
    # Un solo job: compatible con el limite de 1 job por ejecucion en IonQ
    res = StatevectorSampler().run([circ], shots=shots).result()[0]
    return res.data.meas.get_counts()

# ---------------- 6. Ejecutar y comparar ----------------
def pct(v):
    return f"{v} mil ({100 * v / POB_TOTAL:.1f}%)"

if __name__ == "__main__":
    g, e = greedy(), exacta()
    v_opt = poblacion_cubierta(e)
    print(f"Poblacion total: {POB_TOTAL} mil | {N} sitios | {K} ambulancias\n")
    print(f"Greedy : sitios {g} -> {pct(poblacion_cubierta(g))}")
    print(f"Exacta : sitios {e} -> {pct(v_opt)}")

    params = optimizar()
    counts = muestrear(params)
    shots = sum(counts.values())
    mejor = max(counts, key=lambda b: poblacion_cubierta(bits_a_sitios(b)))
    q = bits_a_sitios(mejor)
    print(f"QAOA   : sitios {q} -> {pct(poblacion_cubierta(q))}\n")

    validas = sum(c for b, c in counts.items() if b.count("1") == K) / shots
    p_opt = sum(c for b, c in counts.items()
                if poblacion_cubierta(bits_a_sitios(b)) == v_opt) / shots
    n_opt = sum(1 for s in itertools.combinations(range(N), K)
                if poblacion_cubierta(list(s)) == v_opt)
    azar = n_opt / math.comb(N, K)
    print(f"Mediciones con exactamente {K} ambulancias: {100 * validas:.1f}%")
    print(f"Prob. de medir una solucion optima: {100 * p_opt:.1f}% "
          f"(eligiendo al azar: {100 * azar:.1f}%, {p_opt / azar:.1f}x mejor)")
    print("\nTop 5 resultados medidos:")
    for b, c in sorted(counts.items(), key=lambda kv: -kv[1])[:5]:
        s = bits_a_sitios(b)
        print(f"  sitios {s}: {100 * c / shots:.1f}% de las mediciones, "
              f"cubre {pct(poblacion_cubierta(s))}")
```